

HW 1

Sepehr Ilami

September 2025

Github Link: https://github.com/sepehrilami/phys7332_fa25/blob/main/assignments/assignment01.ipynb

Question 1

a

.csv:

This format is very simple and very well known. It's also readable by humans and works well as supplementary materials of a project/paper. Also, it's easy to import/export in Python and very easy to work with

.pkl:

This is a different format that is more efficient in terms of space and speed. It also keeps the object structure intact, which is useful for complex data types.

.gml:

It perfectly stores nodes and edges and their attributes all together in one file. It's specifically designed for graph data, making it a natural choice for this kind of work. It's also human readable.

ElasticSearch:

Very fast for searching, querying, and indexing huge datasets. It also supports complex queries, making it a powerful tool for data analysis. Great for attribute-based searches.

b

Basically, each format's disadvantages are the advantages of the others. :D

.csv:

while CSV files are simple and human-readable, they are not efficient for large datasets and do not preserve complex data structures, lacking schema or type information. Also, implementing multigraphs and hyperedges can be challenging due to the need for extra columns, and the graph structure is not explicit, requiring additional processing to reconstruct the graph.

.pkl:

I think it's only used with Python, so it's not very portable. Also, it's not human-readable, which can make debugging and manual inspection difficult. Additionally, security concerns arise when unpickling data from untrusted sources, as it can execute arbitrary code.

.gml:

While GML is human-readable and specifically designed for graph data, it can become complicated for very large graphs due to its text-based nature, leading to larger file sizes and slower parsing times. Additionally, GML may not support all complex graph structures natively, such as multigraphs or hyperedges, requiring workarounds. It's main problem though is its parse time and file size for large graphs.

ElasticSearch:

ElasticSearch is not a file format but a search engine, which means it requires setting up and maintaining a server, adding complexity compared to simple file formats. It is probably overkill for small datasets or simple graph structures, and its performance can degrade with very large datasets if not properly managed. You should simulate a graph with queries, which is not as straightforward as using a dedicated graph format.

C

I believe that .pkl files are the ones that pose a security risk. This is because they can execute arbitrary code during the unpickling process, which can be exploited if the data comes from an untrusted source. This makes them potentially dangerous if not handled carefully.

Imagine you receive a package from someone you don't know. If the package contains a simple letter (like a .csv file), you can easily read it without any worries. But if the package contains a complex gadget (like a .pkl file), it could be very dangerous. The pickle can serialize and deserialize not just data, but also executable code in the form of bytecode. So when you unpickle data, you are basically asking the Python interpreter to execute a stream of instructions, including functions, classes, etc. An attacker can use this functionality to create a payload that performs malicious actions upon deserialization. For example, they could craft a pickle that, when unpickled, deletes files, steals data, or opens a backdoor on your system. This is why it's crucial to only unpickle data from trusted sources.

d

Ok so, when each of these formats are ideal to use?

CSV is sharable, readable, simple, and easy to use, making it great for small to medium datasets. It's particularly useful when you want to share data with others or include it as supplementary material in a publication. You can easily attach an edge list along with a node table (attributes), and you're done. Additionally, CSV is excellent for quick prototyping and analysis, especially when working with tabular data.

Pickle is optimal for Python-only workflows, especially when you run an expensive pipeline to compute centralities or communities on a NetworkX graph. It allows you to checkpoint the enriched graph for quick reloads the next day. Its one-line save/load feature preserves all Python attributes exactly and is fast during development. However, it should not be shared externally, as it is best treated as a local cache artifact.

GML is ideal for preparing a graph with node and edge attributes that you want collaborators to open in tools like Gephi. It captures both structure and attributes in a single text file, making it human-inspectable and well-supported by various graph tools.

Elasticsearch is awesome when analyzing massive, time-evolving interaction or telemetry graphs, such as billions of events (e.g., user-item clicks, login IP connections). It provides near-real-time filtering by time window and attributes, along with dashboards that spotlight emerging connections. Its distributed ingestion, fast aggregations, and powerful query Domain-specific Language (DSL) make it suitable for operational large-graph analytics.

Question 2

a

This algorithm will fall in an infinite loop in any complete network. because this if loop would always be true:

```
if has edge(G, i, v) or has edge(G, j, u) then  
    continue
```

because in a complete graph, every node is connected to every other node. So, for any pair of edges (i, j) and (u, v), there will always be an edge between i and v or between j and u. This means that the condition in the if statement will always be true, causing the algorithm to skip the rest of the loop and continue to the next iteration indefinitely.

Some other non-complete graphs that can also cause this algorithm to fall into an infinite loop are:

1. **Star Graphs:** In a star graph, there is a central node connected to all other peripheral nodes. If the algorithm selects edges that involve the central node, it can keep selecting edges that connect to the central node, leading to an infinite loop. Again, based on this line:

```
if i = u or i = v or j = u or j = v then  
    continue
```

1. **Graphs with 3 or fewer nodes:** In such small graphs, the limited number of edges can lead to situations where the algorithm keeps selecting edges that connect the same nodes, causing it to loop indefinitely. This happens based on this line:

```
if i = u or i = v or j = u or j = v then  
    continue
```

1. **Bipartite Graphs with Complete Connections:** In a bipartite graph where every node in one set is connected to every node in the other set, the algorithm can keep selecting edges that connect nodes from different sets, leading to an infinite loop.

b

Instead of just counting successful exchanges with `s`, we can also count the number of attempts with a new variable `attempts`. If the number of attempts exceeds a certain threshold (e.g., `max_attempts`), we can break out of the loop to prevent an infinite loop. There are two different ways to set this threshold: 1) per exchange, or 2) overall. I will implement the first one here; But we can also set a threshold for the overall attempts. This version could be done by adding a new variable `overall_attempts` and checking it against a `max_overall_attempts` threshold, or, more simply, we can use a for loop instead of a while loop for the overall attempts.

```
def edge_exchange(G, t, max_attempts):
    s = 0
    attempts = 0
    while s < t and attempts < max_attempts:
        i, j = get_random_edge(G)
        u, v = get_random_edge(G)
        if i == u or i == v or j == u or j == v:
            attempts += 1
            continue
        if has_edge(G, i, v) or has_edge(G, j, u):
            attempts += 1
            continue
        remove_edge(G, i, j)
        remove_edge(G, u, v)
        add_edge(G, i, v)
        add_edge(G, j, u)
        s += 1 # Reset attempts after a successful exchange
        attempts = 0
```

c

To preserve the overall connectivity of the network during edge exchanges, we need to ensure that the graph remains connected after each exchange. This means that we cannot perform an edge exchange if it would result in a disconnected graph.

This additional constraint will make the exchange more difficult because every time we want to exchange two edges, we need to check if the resulting graph is still connected. This requires additional computations, such as performing a connectivity check (e.g., using DFS/BFS, or even a simple networkx check) after each potential exchange. This added complexity can significantly increase the time complexity of the algorithm, especially for large graphs.

Here is a modified version of the algorithm that includes a connectivity check:

```
import random
import networkx as nx

def connected(G):
    # we can check connectivity using NetworkX, or implement our own
```

DFS/BFS

```
# DFS implementation, for example:
from collections import deque
visited = set()
start_node = next(iter(G.nodes)) # Get an arbitrary starting node
queue = deque([start_node])
while queue:
    node = queue.popleft()
    if node not in visited:
        visited.add(node)
        queue.extend(neighbor for neighbor in G.neighbors(node) if
neighbor not in visited)
return len(visited) == len(G.nodes)
# Alternatively, using networkx's built-in function:
return nx.is_connected(G)

def edge_exchange_with_connectivity(G, max_attempts):
    s = 0
    attempts = 0
    while attempts < max_attempts:
        # Randomly select two edges (i, j) and (u, v) from the graph G
        i, j = get_random_edge(G)
        u, v = get_random_edge(G)
        if i == u or i == v or j == u or j == v:
            attempts += 1
            continue
        if has_edge(G, i, v) or has_edge(G, j, u):
            attempts += 1
            continue
        # Perform the edge exchange
        G.remove_edge(i, j)
        G.remove_edge(u, v)
        G.add_edge(i, v)
        G.add_edge(j, u)
        # Check if the graph is still connected
        if not connected(G):
            # If not connected, revert the changes
            G.remove_edge(i, v)
            G.remove_edge(j, u)
            G.add_edge(i, j)
            G.add_edge(u, v)
            attempts += 1
            continue
        s += 1
        attempts = 0 # Reset attempts after a successful exchange
    return s
```

d

It means that on top of all checks above, now we need to make sure that the new edge preserves the attribute as well. In other words, if i was previously connected to j and now we want to create edge of i to v , we need to make sure that nodes j and v have the same attribute value. This adds another layer of complexity to the edge exchange process, as we now need to check the attributes of the nodes involved in the exchange. This can further reduce the number of valid exchanges that can be made.

Intuitively, I think the expected number of attempts to find a valid exchange will be multiplied by 4. Because for swapping (i,j) and (u,v) to (i,v) and (u,j) , we need to make sure that: 1) i and v have the same attribute, and 2) u and j have the same attribute. Having the same attribute out of two options $\{-1, 1\}$ has a probability of 0.5 (Assuming uniform distribution). So the probability of both conditions being true is $0.5 * 0.5 = 0.25$. Therefore, the expected number of attempts to find a valid exchange will be multiplied by 4.

Suppose your network has a binary attribute $x_i \in \{-1, 1\}$ associated with every node i . Modify the algorithm to preserve the correlation on edges of x . This means that any new edge (a, b) inserted in place of an existing edge (i, j) must have the same attributes $(x_i, x_j) = (x_a, x_b)$.

Here's the modified algorithm:

```
def edge_exchange_with_connectivity_and_attribute(G, max_attempts,
max_overall_attempts):
    s = 0
    attempts = 0
    overall_attempts = 0
    while s < max_attempts and overall_attempts <
max_overall_attempts:
        overall_attempts += 1
        i, j = get_random_edge(G)
        u, v = get_random_edge(G)
        if i == u or i == v or j == u or j == v:
            continue
        if has_edge(G, i, v) or has_edge(G, j, u):
            continue
        if attribute(i) != attribute(v) or attribute(j) !=
attribute(u):
            continue
        attempts = 0
        while attempts < max_attempts:
            attempts += 1
            G_prime = G.copy() # Create a copy of G
            G_prime.remove_edge(i, j)
            G_prime.remove_edge(u, v)
            G_prime.add_edge(i, v)
            G_prime.add_edge(j, u)
            if is_connected(G_prime):
                G = G_prime
                s += 1
```

```
        break
    return G
```

Question 3

a

```
# implement betweenness centrality using Brandes' algorithm, and
compare with networkx
import networkx as nx
import random
import time
from collections import defaultdict, deque
import numpy as np
import matplotlib.pyplot as plt

def generate_random_graph(num_nodes, num_edges):
    G = nx.gnm_random_graph(num_nodes, num_edges)
    return G

def all_shortest_paths(G, start, end):
    if start == end:
        return [[start]]

    # BFS to find shortest distances and parents (We can do DFS as
well)
    distances = {start: 0}
    parents = defaultdict(list)
    queue = deque([start])

    while queue:
        vertex = queue.popleft()
        for neighbor in G.neighbors(vertex):
            if neighbor not in distances:
                distances[neighbor] = distances[vertex] + 1
                queue.append(neighbor)
                parents[neighbor].append(vertex)
            elif distances[neighbor] == distances[vertex] + 1:
                # Another shortest path to this neighbor
                parents[neighbor].append(vertex)

    if end not in distances:
        return [] # No path exists

    # Reconstruct all shortest paths
    def build_paths(node):
        if node == start:
            return [[start]]
```

```

        paths = []
        for parent in parents[node]:
            for path in build_paths(parent):
                paths.append(path + [node])
        return paths

    return build_paths(end)

def calc_betweenness_single_node(G, node):
    nodes = list(G.nodes())
    nodes.remove(node)
    betweenness = 0.0

    for i in range(len(nodes)):
        for j in range(i + 1, len(nodes)):
            s = nodes[i]
            t = nodes[j]
            paths = all_shortest_paths(G, s, t)
            if not paths:
                continue
            count_paths_through_node = sum(1 for path in paths if node
in path[1:-1])
            betweenness += count_paths_through_node / len(paths)

    return betweenness

num_nodes = 10 # Number of nodes
num_edges = 40 # Number of edges
# create a G using networkx
G = generate_random_graph(num_nodes, num_edges)
node = np.random.choice(G.nodes())
print(f'node: {node}')

# function should take the graph and one node, and output the
betweenness centrality of that node
betweenness = calc_betweenness_single_node(G, node)
print(f'betweenness (custom): {betweenness}')

node: 5
betweenness (custom): 0.25

```

b

```

def calc_betweenness_single_node(G, node):
    nodes = list(G.nodes())
    # check if node is in G.nodes
    if node not in nodes:
        raise ValueError(f"Node {node} not in graph")
    # if the graph is not a networkX object, raise an error
    if not isinstance(G, nx.Graph):

```

```

        raise ValueError("G must be a networkx Graph object")
    # check the graph connectivity
    if not nx.is_connected(G):
        raise ValueError("Graph must be connected")
    # if the number of edges are extremely low, raise a warning
    if G.number_of_edges() < 10:
        raise Warning("Graph is very sparse, results may be
inaccurate")
    nodes.remove(node)
    betweenness = 0.0

    for i in range(len(nodes)):
        for j in range(i + 1, len(nodes)):
            s = nodes[i]
            t = nodes[j]
            paths = all_shortest_paths(G, s, t)
            if len(paths) == 0:
                raise ValueError(f"No path between {s} and {t}; graph
must be connected")
            if not paths:
                continue
            count_paths_through_node = sum(1 for path in paths if node
in path[1:-1])
            betweenness += count_paths_through_node / len(paths)

    if betweenness < 0:
        raise ValueError("Betweenness centrality cannot be negative")
    return betweenness

num_nodes = 10 # Number of nodes
num_edges = 40 # Number of edges
# create a G using networkx
G = generate_random_graph(num_nodes, num_edges)
node = np.random.choice(G.nodes())
print(f'node: {node}')

# function should take the graph and one node, and output the
betweenness centrality of that node
betweenness = calc_betweenness_single_node(G, node)
print(f'betweenness (custom): {betweenness}')

node: 7
betweenness (custom): 0.7857142857142857

```

C

```

import networkx as nx
import random

# Challenging test cases

```

```

Graphs = []

# 1. Large dense graph
G1 = nx.complete_graph(20) # Fully connected graph with 20 nodes
Graphs.append(G1)

# 2. Large sparse graph
G2 = nx.gnm_random_graph(100, 150) # 100 nodes, 150 edges
Graphs.append(G2)

# 3. Graph with bridges
G3 = nx.barbell_graph(10, 5) # Two cliques of size 10 connected by a
path of length 5
Graphs.append(G3)

# 4. Graph with high-degree nodes
G4 = nx.star_graph(50) # Star graph with 50 leaves
Graphs.append(G4)

# 5. Random graph
G5 = nx.erdos_renyi_graph(50, 0.1) # Random graph with 50 nodes and
edge probability 0.1
Graphs.append(G5)

# 6. Weighted graph
G6 = nx.gnm_random_graph(30, 60)
for (u, v) in G6.edges():
    G6[u][v]['weight'] = random.randint(1, 10) # Assign random
weights to edges
Graphs.append(G6)

# 7. Large cycle graph
G7 = nx.cycle_graph(100) # Cycle graph with 100 nodes
Graphs.append(G7)

# Testing
for idx, G in enumerate(Graphs):
    print(f"\nTesting Graph {idx + 1}")
    for node in G.nodes():
        try:
            networkx_betweenness = nx.betweenness_centrality(G,
normalized=False)[node]
            custom_betweenness = calc_betweenness_single_node(G, node)
            if abs(networkx_betweenness - custom_betweenness) > 1e-5:
                print(f"Discrepancy for node {node}: Custom =
{custom_betweenness}, NetworkX = {networkx_betweenness}")
        except Exception as e:
            print(f"Error: {e} for node {node} in Graph {idx + 1}. The
NetworkX betweenness is {networkx_betweenness} if
'networkx_betweenness' in locals() else 'N/A'")

```

Testing Graph 1

Testing Graph 2

Error: Graph must be connected for node 0 in Graph 2. The NetworkX betweenness is 738.8126389949916

Error: Graph must be connected for node 1 in Graph 2. The NetworkX betweenness is 171.5019841269841

Error: Graph must be connected for node 2 in Graph 2. The NetworkX betweenness is 0.0

Error: Graph must be connected for node 3 in Graph 2. The NetworkX betweenness is 141.33333333333331

Error: Graph must be connected for node 4 in Graph 2. The NetworkX betweenness is 218.87200364994484

Error: Graph must be connected for node 5 in Graph 2. The NetworkX betweenness is 16.436219336219338

Error: Graph must be connected for node 6 in Graph 2. The NetworkX betweenness is 87.5

Error: Graph must be connected for node 7 in Graph 2. The NetworkX betweenness is 23.28571428571429

Error: Graph must be connected for node 8 in Graph 2. The NetworkX betweenness is 125.61666666666667

Error: Graph must be connected for node 9 in Graph 2. The NetworkX betweenness is 280.72136278753914

Error: Graph must be connected for node 10 in Graph 2. The NetworkX betweenness is 114.50647865206686

Error: Graph must be connected for node 11 in Graph 2. The NetworkX betweenness is 221.39999999999995

Error: Graph must be connected for node 12 in Graph 2. The NetworkX betweenness is 303.7019841269841

Error: Graph must be connected for node 13 in Graph 2. The NetworkX betweenness is 48.53888888888889

Error: Graph must be connected for node 14 in Graph 2. The NetworkX betweenness is 0.0

Error: Graph must be connected for node 15 in Graph 2. The NetworkX betweenness is 221.83936423054067

Error: Graph must be connected for node 16 in Graph 2. The NetworkX betweenness is 98.84949494949501

Error: Graph must be connected for node 17 in Graph 2. The NetworkX betweenness is 235.4491405653171

Error: Graph must be connected for node 18 in Graph 2. The NetworkX betweenness is 143.18419064595537

Error: Graph must be connected for node 19 in Graph 2. The NetworkX betweenness is 53.58333333333332

Error: Graph must be connected for node 20 in Graph 2. The NetworkX betweenness is 120.33253968253968

Error: Graph must be connected for node 21 in Graph 2. The NetworkX betweenness is 252.2679865885747

Error: Graph must be connected for node 22 in Graph 2. The NetworkX betweenness is 0.0

Error: Graph must be connected for node 23 in Graph 2. The NetworkX betweenness is 251.4483108394873

Error: Graph must be connected for node 24 in Graph 2. The NetworkX betweenness is 133.15952380952382

Error: Graph must be connected for node 25 in Graph 2. The NetworkX betweenness is 0.0

Error: Graph must be connected for node 26 in Graph 2. The NetworkX betweenness is 0.0

Error: Graph must be connected for node 27 in Graph 2. The NetworkX betweenness is 29.033333333333333

Error: Graph must be connected for node 28 in Graph 2. The NetworkX betweenness is 94.0

Error: Graph must be connected for node 29 in Graph 2. The NetworkX betweenness is 239.6327052032934

Error: Graph must be connected for node 30 in Graph 2. The NetworkX betweenness is 177.16097317714966

Error: Graph must be connected for node 31 in Graph 2. The NetworkX betweenness is 240.74444444444455

Error: Graph must be connected for node 32 in Graph 2. The NetworkX betweenness is 0.0

Error: Graph must be connected for node 33 in Graph 2. The NetworkX betweenness is 0.0

Error: Graph must be connected for node 34 in Graph 2. The NetworkX betweenness is 159.59682539682538

Error: Graph must be connected for node 35 in Graph 2. The NetworkX betweenness is 0.0

Error: Graph must be connected for node 36 in Graph 2. The NetworkX betweenness is 222.521021135727

Error: Graph must be connected for node 37 in Graph 2. The NetworkX betweenness is 0.0

Error: Graph must be connected for node 38 in Graph 2. The NetworkX betweenness is 101.00000000000001

Error: Graph must be connected for node 39 in Graph 2. The NetworkX betweenness is 55.02222222222221

Error: Graph must be connected for node 40 in Graph 2. The NetworkX betweenness is 747.7207070707068

Error: Graph must be connected for node 41 in Graph 2. The NetworkX betweenness is 0.0

Error: Graph must be connected for node 42 in Graph 2. The NetworkX betweenness is 130.67595917154748

Error: Graph must be connected for node 43 in Graph 2. The NetworkX betweenness is 181.99024488583308

Error: Graph must be connected for node 44 in Graph 2. The NetworkX betweenness is 34.27549019607844

Error: Graph must be connected for node 45 in Graph 2. The NetworkX betweenness is 273.4850861556744

Error: Graph must be connected for node 46 in Graph 2. The NetworkX betweenness is 43.59087301587302

Error: Graph must be connected for node 47 in Graph 2. The NetworkX betweenness is 175.34948009506826

Error: Graph must be connected for node 48 in Graph 2. The NetworkX betweenness is 0.0
Error: Graph must be connected for node 49 in Graph 2. The NetworkX betweenness is 228.33333333333334
Error: Graph must be connected for node 50 in Graph 2. The NetworkX betweenness is 52.67857142857143
Error: Graph must be connected for node 51 in Graph 2. The NetworkX betweenness is 88.44041040658688
Error: Graph must be connected for node 52 in Graph 2. The NetworkX betweenness is 0.0
Error: Graph must be connected for node 53 in Graph 2. The NetworkX betweenness is 0.0
Error: Graph must be connected for node 54 in Graph 2. The NetworkX betweenness is 25.083333333333334
Error: Graph must be connected for node 55 in Graph 2. The NetworkX betweenness is 177.59362108479755
Error: Graph must be connected for node 56 in Graph 2. The NetworkX betweenness is 502.3345238095237
Error: Graph must be connected for node 57 in Graph 2. The NetworkX betweenness is 280.557796451914
Error: Graph must be connected for node 58 in Graph 2. The NetworkX betweenness is 119.37116119174945
Error: Graph must be connected for node 59 in Graph 2. The NetworkX betweenness is 110.88412698412701
Error: Graph must be connected for node 60 in Graph 2. The NetworkX betweenness is 185.8907350819115
Error: Graph must be connected for node 61 in Graph 2. The NetworkX betweenness is 45.87460317460317
Error: Graph must be connected for node 62 in Graph 2. The NetworkX betweenness is 448.6720185043714
Error: Graph must be connected for node 63 in Graph 2. The NetworkX betweenness is 275.36382310499965
Error: Graph must be connected for node 64 in Graph 2. The NetworkX betweenness is 0.0
Error: Graph must be connected for node 65 in Graph 2. The NetworkX betweenness is 65.36944444444445
Error: Graph must be connected for node 66 in Graph 2. The NetworkX betweenness is 200.1953314659197
Error: Graph must be connected for node 67 in Graph 2. The NetworkX betweenness is 21.905288175876407
Error: Graph must be connected for node 68 in Graph 2. The NetworkX betweenness is 240.97499999999994
Error: Graph must be connected for node 69 in Graph 2. The NetworkX betweenness is 0.0
Error: Graph must be connected for node 70 in Graph 2. The NetworkX betweenness is 0.0
Error: Graph must be connected for node 71 in Graph 2. The NetworkX betweenness is 26.738492063492068
Error: Graph must be connected for node 72 in Graph 2. The NetworkX betweenness is 401.66666666666674

Error: Graph must be connected for node 73 in Graph 2. The NetworkX betweenness is 25.504761904761896
Error: Graph must be connected for node 74 in Graph 2. The NetworkX betweenness is 666.5013835837367
Error: Graph must be connected for node 75 in Graph 2. The NetworkX betweenness is 0.0
Error: Graph must be connected for node 76 in Graph 2. The NetworkX betweenness is 107.48177149647739
Error: Graph must be connected for node 77 in Graph 2. The NetworkX betweenness is 41.91564171122995
Error: Graph must be connected for node 78 in Graph 2. The NetworkX betweenness is 15.84285714285714
Error: Graph must be connected for node 79 in Graph 2. The NetworkX betweenness is 186.0
Error: Graph must be connected for node 80 in Graph 2. The NetworkX betweenness is 102.83621933621934
Error: Graph must be connected for node 81 in Graph 2. The NetworkX betweenness is 162.3091991341991
Error: Graph must be connected for node 82 in Graph 2. The NetworkX betweenness is 127.2922650878533
Error: Graph must be connected for node 83 in Graph 2. The NetworkX betweenness is 265.3333333333334
Error: Graph must be connected for node 84 in Graph 2. The NetworkX betweenness is 0.0
Error: Graph must be connected for node 85 in Graph 2. The NetworkX betweenness is 391.32404719463545
Error: Graph must be connected for node 86 in Graph 2. The NetworkX betweenness is 218.66167345726166
Error: Graph must be connected for node 87 in Graph 2. The NetworkX betweenness is 75.77790722349545
Error: Graph must be connected for node 88 in Graph 2. The NetworkX betweenness is 100.59325396825402
Error: Graph must be connected for node 89 in Graph 2. The NetworkX betweenness is 444.63535141329254
Error: Graph must be connected for node 90 in Graph 2. The NetworkX betweenness is 55.93333333333332
Error: Graph must be connected for node 91 in Graph 2. The NetworkX betweenness is 94.0
Error: Graph must be connected for node 92 in Graph 2. The NetworkX betweenness is 31.096197266785502
Error: Graph must be connected for node 93 in Graph 2. The NetworkX betweenness is 270.0428571428572
Error: Graph must be connected for node 94 in Graph 2. The NetworkX betweenness is 114.53849206349209
Error: Graph must be connected for node 95 in Graph 2. The NetworkX betweenness is 65.65873015873015
Error: Graph must be connected for node 96 in Graph 2. The NetworkX betweenness is 0.0
Error: Graph must be connected for node 97 in Graph 2. The NetworkX betweenness is 0.0

Error: Graph must be connected for node 98 in Graph 2. The NetworkX betweenness is 140.54523809523815
Error: Graph must be connected for node 99 in Graph 2. The NetworkX betweenness is 408.1066802478568

Testing Graph 3

Testing Graph 4

Testing Graph 5

Testing Graph 6

Error: Graph must be connected for node 0 in Graph 6. The NetworkX betweenness is 51.935497835497834
Error: Graph must be connected for node 1 in Graph 6. The NetworkX betweenness is 50.884199134199136
Error: Graph must be connected for node 2 in Graph 6. The NetworkX betweenness is 9.583333333333334
Error: Graph must be connected for node 3 in Graph 6. The NetworkX betweenness is 3.95
Error: Graph must be connected for node 4 in Graph 6. The NetworkX betweenness is 13.882251082251079
Error: Graph must be connected for node 5 in Graph 6. The NetworkX betweenness is 0.0
Error: Graph must be connected for node 6 in Graph 6. The NetworkX betweenness is 21.37424242424242
Error: Graph must be connected for node 7 in Graph 6. The NetworkX betweenness is 9.376190476190477
Error: Graph must be connected for node 8 in Graph 6. The NetworkX betweenness is 2.5333333333333333
Error: Graph must be connected for node 9 in Graph 6. The NetworkX betweenness is 8.807575757575757
Error: Graph must be connected for node 10 in Graph 6. The NetworkX betweenness is 21.63030303030303
Error: Graph must be connected for node 11 in Graph 6. The NetworkX betweenness is 4.857575757575757
Error: Graph must be connected for node 12 in Graph 6. The NetworkX betweenness is 15.640909090909087
Error: Graph must be connected for node 13 in Graph 6. The NetworkX betweenness is 38.1
Error: Graph must be connected for node 14 in Graph 6. The NetworkX betweenness is 1.8
Error: Graph must be connected for node 15 in Graph 6. The NetworkX betweenness is 41.556493506493496
Error: Graph must be connected for node 16 in Graph 6. The NetworkX betweenness is 5.285714285714286
Error: Graph must be connected for node 17 in Graph 6. The NetworkX betweenness is 11.633333333333333
Error: Graph must be connected for node 18 in Graph 6. The NetworkX betweenness is 23.61190476190476

```
Error: Graph must be connected for node 19 in Graph 6. The NetworkX
betweenness is 9.142857142857144
Error: Graph must be connected for node 20 in Graph 6. The NetworkX
betweenness is 29.474242424242426
Error: Graph must be connected for node 21 in Graph 6. The NetworkX
betweenness is 5.6833333333333333
Error: Graph must be connected for node 22 in Graph 6. The NetworkX
betweenness is 5.526190476190476
Error: Graph must be connected for node 23 in Graph 6. The NetworkX
betweenness is 12.806060606060607
Error: Graph must be connected for node 24 in Graph 6. The NetworkX
betweenness is 21.400432900432904
Error: Graph must be connected for node 25 in Graph 6. The NetworkX
betweenness is 4.542857142857144
Error: Graph must be connected for node 26 in Graph 6. The NetworkX
betweenness is 20.349999999999994
Error: Graph must be connected for node 27 in Graph 6. The NetworkX
betweenness is 62.02510822510823
Error: Graph must be connected for node 28 in Graph 6. The NetworkX
betweenness is 0.0
Error: Graph must be connected for node 29 in Graph 6. The NetworkX
betweenness is 32.606060606060606
```

Testing Graph 7

If I use my function from part a, I will get the exact same betweenness values as networkx. However, if I use the function from part b, I will get errors, while networkx will still give some values. This is because in part b, I handled errors in case of disconnected graphs and other edge cases.

Oh and by the way, the networkx function should not be normalized. If you don't use `normalized=False`, you will get different values because the default is `True`.

d

Run the code below to have a graph that computes the betweenness centrality even for disconnected graphs

```
# Graph that computes the betweenness centrality even for disconnected
graphs

def calc_betweenness_single_node(G, node):
    nodes = list(G.nodes())
    # Check if node is in G.nodes
    if node not in nodes:
        raise ValueError(f"Node {node} not in graph")
    # If the graph is not a networkX object, raise an error
    if not isinstance(G, nx.Graph):
        raise ValueError("G must be a networkx Graph object")
```

```

# Initialize betweenness centrality
betweenness = 0.0

# Get connected components of the graph
connected_components = list(nx.connected_components(G))

# Iterate over each connected component
for component in connected_components:
    if node not in component:
        # Skip components that do not contain the target node
        continue

    # Create a subgraph for the current connected component
    subgraph = G.subgraph(component)
    component_nodes = list(subgraph.nodes())
    component_nodes.remove(node)

    # Compute betweenness centrality for the node within this
    component
    for i in range(len(component_nodes)):
        for j in range(i + 1, len(component_nodes)):
            s = component_nodes[i]
            t = component_nodes[j]
            paths = list(nx.all_shortest_paths(subgraph, s, t))
            if not paths:
                continue
            count_paths_through_node = sum(1 for path in paths if
node in path[1:-1])
            betweenness += count_paths_through_node / len(paths)

    # Ensure betweenness centrality is non-negative
    if betweenness < 0:
        raise ValueError("Betweenness centrality cannot be negative")

    return betweenness

import networkx as nx
import numpy as np

def test_centrality_function(centrality_func):
    # List of challenging graphs
    graphs = []

    # 1. Empty graph
    G1 = nx.Graph()
    graphs.append(("Empty graph", G1))

    # 2. Graph with a single node
    G2 = nx.Graph()

```

```

G2.add_node(0)
graphs.append(("Single node graph", G2))

# 3. Disconnected graph
G3 = nx.Graph()
G3.add_nodes_from([0, 1, 2])
graphs.append(("Disconnected graph", G3))

# 4. Graph with one edge
G4 = nx.Graph()
G4.add_edge(0, 1)
graphs.append(("Graph with one edge", G4))

# 5. Dense graph
G5 = nx.complete_graph(5)
graphs.append(("Dense graph", G5))

# 6. Weighted graph
G6 = nx.Graph()
G6.add_edge(0, 1, weight=2)
G6.add_edge(1, 2, weight=3)
G6.add_edge(2, 3, weight=4)
graphs.append(("Weighted graph", G6))

# 7. Large sparse graph
G7 = nx.gnm_random_graph(100, 150)
graphs.append(("Large sparse graph", G7))

# 8. Large dense graph
G8 = nx.complete_graph(20)
graphs.append(("Large dense graph", G8))

# 9. Graph with bridges
G9 = nx.barbell_graph(10, 5)
graphs.append(("Graph with bridges", G9))

# 10. Star graph
G10 = nx.star_graph(50)
graphs.append(("Star graph", G10))

# Run tests
for name, G in graphs:
    print(f"\nTesting: {name}")
    # Compute NetworkX's betweenness centrality for all nodes
    nx_betweenness = nx.betweenness_centrality(G,
normalized=False)

    for node in G.nodes():
        try:
            # print(f"Testing node {node} in {name}")

```

```

        # Compute the result using the custom function
        custom_result = centrality_func(G, node)
        # print(f"Custom result for node {node}:
{custom_result}")

        # Compare with NetworkX's result
        nx_result = nx_betweenness[node]
        # print(f"NetworkX result for node {node}:
{nx_result}")

        # Check if the results are close
        if abs(custom_result - nx_result) > 1e-5:
            print(f"Discrepancy found for node {node}:
Custom={custom_result}, NetworkX={nx_result}")
        # else:
        #     print(f"Results match for node {node}")
    except Exception as e:
        print(f"Error in {name}: {e}")

# Testing our betweenness centrality function
test_centrality_function(calc_betweenness_single_node)

```

Testing: Empty graph

Testing: Single node graph

Testing: Disconnected graph

Testing: Graph with one edge

Testing: Dense graph

Testing: Weighted graph

Testing: Large sparse graph

Testing: Large dense graph

Testing: Graph with bridges

Testing: Star graph

These test cases cover a wide range of scenarios, including edge cases, small and large graphs, sparse and dense graphs, weighted graphs, and graphs with specific structural features like bridges and hubs. This ensures the function is robust, accurate, and performs well under different conditions. Here's why each test case was included:

1. Empty Graph:
 - Tests the function's behavior when there are no nodes or edges.

2. Single Node Graph:
 - Tests the function's behavior with the simplest possible graph containing only one node.
 - The centrality should be zero since there are no paths.
3. Disconnected Graph:
 - Tests how the function handles graphs with multiple nodes but no edges.
 - Ensures the function correctly calculates zero centrality for all nodes.
4. Graph with One Edge:
 - A minimal connected graph with two nodes and one edge.
 - Tests the function's ability to calculate centrality in a simple connected graph.
5. Dense Graph:
 - A complete graph where every node is connected to every other node.
 - Tests the function's performance and correctness in a highly connected graph.
6. Weighted Graph:
 - Includes edge weights to test if the function can handle weighted graphs correctly.
 - Ensures the function considers weights when calculating centrality.
7. Large Sparse Graph:
 - Tests the function's scalability and correctness in sparse graphs.
8. Large Dense Graph:
 - A complete graph with a larger number of nodes.
 - Tests the function's performance and correctness in dense graphs with more nodes.
9. Graph with Bridges:
 - A barbell graph with two cliques connected by a bridge.
 - Tests the function's ability to identify nodes with high centrality due to their role as bridges.
10. Star Graph:
 - A graph with one central node connected to all other nodes.
 - Tests if the function correctly identifies the central node as having the highest centrality.

Question 4

a

I'll scrape Industrial Engineering department at Northeastern University.

First, loading course names, IDs, and descriptions..

```
!pip install requests
```

```
Requirement already satisfied: requests in  
/Users/sepehr/miniconda/envs/gametheory/lib/python3.13/site-packages  
(2.32.5)
```

Requirement already satisfied: charset_normalizer<4,>=2 in
/Users/sepehr/miniconda/envs/gametheory/lib/python3.13/site-packages
(from requests) (3.4.3)

Requirement already satisfied: idna<4,>=2.5 in
/Users/sepehr/miniconda/envs/gametheory/lib/python3.13/site-packages
(from requests) (3.10)

Requirement already satisfied: urllib3<3,>=1.21.1 in
/Users/sepehr/miniconda/envs/gametheory/lib/python3.13/site-packages
(from requests) (2.5.0)

Requirement already satisfied: certifi>=2017.4.17 in
/Users/sepehr/miniconda/envs/gametheory/lib/python3.13/site-packages
(from requests) (2025.8.3)

```
# import requests and BeautifulSoup
```

```
import requests
```

```
from bs4 import BeautifulSoup
```

```
# Your Turn!
```

```
def
```

```
get_course_titles_and_descriptions(dept_html='https://catalog.northeastern.edu/course-descriptions/ie/'):
    """
```

```
    Given a string of raw html (dept_html) that contains a
    department's course titles and descriptions,
    return a useful data structure that contains only the titles and
    descriptions.
    """
```

```
    # get the raw text
```

```
    ie_html = requests.get(my_full_url).text
```

```
    soup = BeautifulSoup(ie_html, 'html.parser') # Specify the parser
    soup.prettify()
```

```
    soup_alpha_div = soup.find_all('div', {'class':"courseblock"})
```

```
    course_titles = [] # Initialize the list here
```

```
    for course in soup_alpha_div:
```

```
        title_raw = course.find('p', {'class':'courseblocktitle
noindent'}).text
```

```
        course_titles.append(title_raw) # Append titles to the list
```

```
    return course_titles
```

```
my_full_url =
```

```
'https://catalog.northeastern.edu/course-descriptions/ie/'
```

```
course_titles = get_course_titles_and_descriptions(my_full_url)
```

```
print(list(course_titles))
```

```
['IE\xa01990. Elective. (1-4 Hours)', 'IE\xa02310. Introduction to
Industrial Engineering. (4 Hours)', 'IE\xa02311. Recitation for IE
```

2310. (0 Hours)', 'IE\xa02990. Elective. (1-4 Hours)', 'IE\xa03412. Engineering Probability and Statistics. (4 Hours)', 'IE\xa03425. Engineering Database Systems. (4 Hours)', 'IE\xa03426. Recitation for IE 3425. (0 Hours)', 'IE\xa03500. Introduction to Healthcare Systems Engineering. (4 Hours)', 'IE\xa03990. Elective. (1-4 Hours)', 'IE\xa04510. Simulation Modeling and Analysis. (4 Hours)', 'IE\xa04512. Engineering Economy. (4 Hours)', 'IE\xa04515. Operations Research. (4 Hours)', 'IE\xa04516. Quality Assurance. (4 Hours)', 'IE\xa04520. Stochastic Modeling. (4 Hours)', 'IE\xa04522. Human-Machine Systems. (4 Hours)', 'IE\xa04523. Lab for IE 4522. (1 Hour)', 'IE\xa04525. Logistics and Supply Chain Management. (4 Hours)', 'IE\xa04530. Manufacturing Systems and Techniques. (4 Hours)', 'IE\xa04531. Lab for IE 4530. (1 Hour)', 'IE\xa04535. Human-Machine Systems in a Global Context. (5 Hours)', 'IE\xa04625. Facilities Planning and Material Handling. (4 Hours)', 'IE\xa04699. Special Topics in Industrial Engineering. (4 Hours)', 'IE\xa04990. Elective. (1-4 Hours)', 'IE\xa04991. Research. (4 Hours)', 'IE\xa05137. Computational Modeling in Industrial Engineering. (4 Hours)', 'IE\xa05200. Mathematics for Machine Learning. (4 Hours)', 'IE\xa05360. Digital Manufacturing. (4 Hours)', 'IE\xa05374. Special Topics in Industrial Engineering. (4 Hours)', 'IE\xa05380. Integrated Automation. (4 Hours)', 'IE\xa05390. Structured Data Analytics for Industrial Engineering. (4 Hours)', 'IE\xa05400. Healthcare Systems Modeling and Analysis. (4 Hours)', 'IE\xa05500. Systems Engineering in Public Programs. (4 Hours)', 'IE\xa05617. Lean Concepts and Applications. (4 Hours)', 'IE\xa05618. Recitation for IE 5617. (0 Hours)', 'IE\xa05630. Biosensor and Human Behavior Measurement. (4 Hours)', 'IE\xa05640. Data Mining for Engineering Applications. (4 Hours)', 'IE\xa05976. Directed Study. (1-4 Hours)', 'IE\xa06200. Engineering Probability and Statistics. (4 Hours)', 'IE\xa06300. Manufacturing Systems Design. (4 Hours)', 'IE\xa06400. Foundations for Data Analytics Engineering. (4 Hours)', 'IE\xa06500. Human Performance. (4 Hours)', 'IE\xa06600. Computation and Visualization for Analytics. (4 Hours)', 'IE\xa06700. Data Management for Analytics. (4 Hours)', 'IE\xa06750. Data Warehousing and Integration. (4 Hours)', 'IE\xa06962. Elective. (1-4 Hours)', 'IE\xa07200. Supply Chain Engineering. (4 Hours)', 'IE\xa07215. Simulation Analysis. (4 Hours)', 'IE\xa07270. Intelligent Manufacturing. (4 Hours)', 'IE\xa07275. Data Mining in Engineering. (4 Hours)', 'IE\xa07280. Statistical Methods in Engineering. (4 Hours)', 'IE\xa07285. Statistical Quality Control. (4 Hours)', 'IE\xa07290. Reliability Analysis and Risk Assessment. (4 Hours)', 'IE\xa07295. Applied Reinforcement Learning in Engineering. (4 Hours)', 'IE\xa07300. Statistical Learning for Engineering. (4 Hours)', 'IE\xa07315. Human Factors Engineering. (4 Hours)', 'IE\xa07350. Sociotechnical Systems: Computational Models for Design and Policy. (4 Hours)', 'IE\xa07374. Special Topics in Industrial Engineering. (4 Hours)', 'IE\xa07440. Industrial Engineering Leadership Challenge Project 1. (4 Hours)', 'IE\xa07442. Industrial Engineering

```
Leadership Challenge Project 2. (4 Hours)', 'IE\xa07500. Applied
Natural Language Processing in Engineering. (4 Hours)', 'IE\xa07615.
Neural Networks and Deep Learning. (4 Hours)', 'IE\xa07945. Master's
Project. (4 Hours)', 'IE\xa07962. Elective. (1-4 Hours)', 'IE\
\xa07976. Directed Study. (1-4 Hours)', 'IE\xa07986. Research. (0
Hours)', 'IE\xa07990. Thesis. (4 Hours)', 'IE\xa07996. Thesis
Continuation - Half-Time. (0 Hours)', 'IE\xa08960. Candidacy
Preparation-Doctoral. (0 Hours)', 'IE\xa08986. Research. (0
Hours)', 'IE\xa09000. PhD Candidacy Achieved. (0 Hours)', 'IE\
\xa09986. Research. (0 Hours)', 'IE\xa09990. Dissertation Term 1.
(0 Hours)', 'IE\xa09991. Dissertation Term 2. (0 Hours)', 'IE\
\xa09996. Dissertation Continuation. (0 Hours)']
```

Now let's scrape all the departments in Northeastern University!

```
import re
import requests
from urllib.parse import urljoin
from bs4 import BeautifulSoup

CATALOG_INDEX = "https://catalog.northeastern.edu/course-
descriptions/"
COURSE_ID_RE = re.compile(r"\b[A-Z]{2,}\s?\d{4}[A-Z]?\b")

def normalize_spaces(s: str):
    return s.replace("\xa0", " ").strip()

def parse_course_id_from_title(title_text: str):
    return normalize_spaces(title_text.split(".", 1)[0])

def extract_course_ids(text: str):
    return [normalize_spaces(m) for m in COURSE_ID_RE.findall(text)]

def get_courses_and_prereqs(dept_url: str):
    """
    Parse ONE department/subject page like:
    https://catalog.northeastern.edu/course-descriptions/ie/
    Returns a list of dicts with course_id, title, prereqs, dept.
    """
    html = requests.get(dept_url).text
    soup = BeautifulSoup(html, "html.parser")

    # try to infer the short dept code from the URL (e.g., 'ie')
    dept_code = dept_url.rstrip("/").split("/")[-1].upper()

    courses = []
    for block in soup.select("div.courseblock"):
        title_p = block.select_one("p.courseblocktitle")
        if not title_p:
            continue
```

```

        full_title = normalize_spaces(title_p.get_text(" ",
strip=True))
        course_id = parse_course_id_from_title(full_title)

        prereq_ids = []
        for p in block.select("p.courseblockextra"):
            strong = p.find("strong")
            if not strong or "Prerequisite" not in strong.get_text():
                continue

            links = [normalize_spaces(a.get_text(strip=True)) for a in
p.find_all("a")]
            text_codes = extract_course_ids(p.get_text(" ",
strip=True))
            prereq_ids = sorted(set(links + text_codes))

            courses.append({
                "dept": dept_code,
                "course_id": course_id,
                "title": full_title,
                "prereqs": prereq_ids,
            })

    return courses

def get_department_urls(index_url: str = CATALOG_INDEX):
    """
    Scrape the catalog index for all department/subject pages under
    /course-descriptions/.
    """
    html = requests.get(index_url).text
    soup = BeautifulSoup(html, "html.parser")

    dept_urls = set()
    for a in soup.select("a[href]"):
        href = a["href"]
        # keep links like /course-descriptions/ie/ (one path segment)
        if re.fullmatch(r"/course-descriptions/[a-z0-9-]+/", href):
            dept_urls.add(urljoin(index_url, href))
    return sorted(dept_urls)

def get_all_courses_and_prereqs(index_url: str = CATALOG_INDEX):
    """
    Scrape EVERY department found on the index page.
    Returns a single list of course dicts (dept, course_id, title,
    prereqs).
    """
    all_data = []
    for dept_url in get_department_urls(index_url):

```

```

    try:
        all_data.extend(get_courses_and_prereqs(dept_url))
    except Exception as e:
        # keep going if one subject page is quirky
        print(f"Skipping {dept_url}: {e}")
        continue
return all_data

all_courses = get_all_courses_and_prereqs()
print(len(all_courses), "courses found")
print(all_courses[:5]) # peek

7967 courses found
[{'dept': 'AACE', 'course_id': 'AACE 1990', 'title': 'AACE 1990.
Elective. (1-4 Hours)', 'prereqs': []}, {'dept': 'AACE', 'course_id':
'AACE 2990', 'title': 'AACE 2990. Elective. (1-4 Hours)', 'prereqs':
[]}, {'dept': 'AACE', 'course_id': 'AACE 3990', 'title': 'AACE 3990.
Elective. (1-4 Hours)', 'prereqs': []}, {'dept': 'AACE', 'course_id':
'AACE 4990', 'title': 'AACE 4990. Elective. (1-4 Hours)', 'prereqs':
[]}, {'dept': 'AACE', 'course_id': 'AACE 6000', 'title': 'AACE 6000.
Arts and Culture Organizational Leadership. (3 Hours)', 'prereqs':
[]}]

```

b

```

# get the list of all department names from all_courses keys
departments = set(course['dept'] for course in all_courses)
print(departments)

{'RGA', 'ARMY', 'ANTH', 'ARTG', 'AFRS', 'GET', 'AMSL', 'AAI', 'PPUA',
'EDUT', 'PSY', 'PHL', 'DAMG', 'CS', 'ABRB', 'ENSY', 'BIOL', 'MSCI',
'HSCI', 'HLTH', 'EET', 'GIS', 'RFA', 'BUSN', 'HBRW', 'THTR', 'SCLY',
'WMNS', 'GST', 'ABRL', 'EESC', 'FIN', 'MUSC', 'CHEM', 'EEAM', 'MISM',
'HST', 'OR', 'ARTH', 'SMFA', 'EDU', 'INTP', 'ITC', 'GE', 'POL',
'ENCP', 'ECNM', 'FRNH', 'PDM', 'MGSC', 'HSV', 'MKT', 'BIO', 'EECE',
'CHM', 'NRSG', 'GAME', 'LING', 'PLSC', 'ME', 'HINF', 'CMN', 'PTH',
'GBST', 'SMT', 'ANT', 'PHLS', 'BIOT', 'MGMT', 'AVM', 'LACS', 'HONR',
'LST', 'HIST', 'CHNS', 'MTH', 'RPT', 'BNSC', 'ENGR', 'CED', 'ARTD',
'INTB', 'SLPA', 'ABRU', 'COOP', 'MET', 'BTC', 'BIOE', 'LDR', 'HLS',
'ARAB', 'AFCS', 'PJM', 'HRM', 'ECON', 'TELE', 'CMMN', 'EDUC', 'CMG',
'EAI', 'IE', 'SBSY', 'MUSI', 'CLTR', 'HRMG', 'ABRH', 'KORE', 'MATH',
'PHMD', 'SPNS', 'MGT', 'POLS', 'GRMN', 'ARTF', 'DGM', 'EXRE', 'HUSV',
'HSC', 'EEBA', 'INNO', 'BIOC', 'ENVS', 'ECN', 'ENTR', 'EXSC', 'FINA',
'PMST', 'MECN', 'APL', 'LANG', 'INFO', 'MUS', 'IS', 'PHIL', 'ENG',
'JRNL', 'SIA', 'ASNS', 'HSTY', 'PHTH', 'ABRS', 'PST', 'COP', 'LAW',
'PA', 'DS', 'CRTE', 'GSND', 'DEAF', 'LARC', 'ORGB', 'INAM', 'FSEM',
'PSYC', 'RSSN', 'HMG', 'DADS', 'INMI', 'AACE', 'LW', 'PORT', 'ACCT',
'INSC', 'ABRC', 'ALY', 'ARTE', 'INSH', 'MATM', 'NTR', 'PT', 'SOC',
'TELR', 'ART', 'LS', 'INT', 'CY', 'STRT', 'CAEP', 'NNMD', 'BINF',
'CSYE', 'EMGT', 'CNET', 'MKTG', 'PHY', 'COMM', 'ACC', 'ARTS', 'NPM',

```

```
'EESH', 'PBR', 'MATL', 'PHSC', 'CHME', 'ENGL', 'PHYS', 'EXED', 'SUEN',
'INS', 'CRWT', 'MSCR', 'PMCL', 'DA', 'CET', 'PHDL', 'ARCH', 'ENLR',
'ESC', 'INTL', 'RMS', 'SCHM', 'TCC', 'CIVE', 'ITLN', 'CRIM', 'DGTR',
'JWSS', 'LWP', 'SOCL', 'MEIE', 'CJS', 'ABRD', 'PREL', 'JPNS', 'NETS',
'ENGW', 'MUST', 'LPSC', 'INPR', 'EEMB', 'ENVR'}
```

keep your existing builder unchanged

```
import networkx as nx
```

```
def build_curriculum_graph(courses):
```

```
    G = nx.DiGraph()
```

```
    for c in courses:
```

```
        cid = c["course_id"]
```

```
        G.add_node(cid, title=c.get("title"), dept=c.get("dept"))
```

```
        for pre in c["prereqs"]:
```

```
            G.add_edge(pre, cid) # prereq -> course
```

```
    return G
```

---- pick a department and filter ----

```
dept = "LAW" # e.g., Industrial Engineering
```

```
dept_courses = [c for c in all_courses if c.get("dept", "").upper() == dept]
```

```
G = build_curriculum_graph(dept_courses)
```

```
print(G.number_of_nodes(), "nodes;", G.number_of_edges(), "edges")
```

```
print("Sample edges:", list(G.edges())[:10])
```

```
214 nodes; 246 edges
```

```
Sample edges: [('LAW 3101', 'LAW 3120'), ('LAW 3101', 'LAW 3130'),
('LAW 3101', 'LAW 3140'), ('LAW 3101', 'LAW 3150'), ('LAW 3101', 'LAW
3160'), ('LAW 3101', 'LAW 3170'), ('LAW 3101', 'LAW 3181'), ('LAW
3101', 'LAW 3182'), ('LAW 3101', 'LAW 3210'), ('LAW 3101', 'LAW
3211')]
```

C

```
from collections import defaultdict
```

aggregate totals per dept

```
dept_totals = defaultdict(lambda: {"courses": 0, "prereqs": 0})
```

```
for c in all_courses:
```

```
    # infer dept from course_id if missing
```

```
    dept = (c.get("dept") or c["course_id"].split()[0]).upper()
```

```
    # unique prereqs per course
```

```
    prereq_count = len(set(c.get("prereqs", [])))
```

```
    # update totals
```

```
    dept_totals[dept]["courses"] += 1
```

```
    dept_totals[dept]["prereqs"] += prereq_count
```

```

# compute averages per dept
avg_prereqs_per_dept = {
    d: totals["prereqs"] / totals["courses"]
    for d, totals in dept_totals.items() if totals["courses"] > 0
}

# report, sorted descending
ranked = sorted(avg_prereqs_per_dept.items(), key=lambda x: x[1],
reverse=True)

print("Average prerequisites per course, by department (descending):")
for d, avg in ranked:
    print(f"{d}: {avg:.2f} prereqs/course (n={dept_totals[d]
['courses']})")

top_dept, top_avg = ranked[0]
print(f"\nDepartment with the most prerequisites per course on
average: {top_dept} ({top_avg:.2f})")

```

Average prerequisites per course, by department (descending):

```

ENGL: 3.47 prereqs/course (n=114)
CRWT: 3.33 prereqs/course (n=18)
ANTH: 3.22 prereqs/course (n=36)
ENGW: 3.16 prereqs/course (n=19)
PSYC: 2.49 prereqs/course (n=97)
FINA: 2.18 prereqs/course (n=115)
ENCP: 2.00 prereqs/course (n=9)
LS: 1.83 prereqs/course (n=24)
MGT: 1.83 prereqs/course (n=18)
SCHM: 1.82 prereqs/course (n=28)
SPNS: 1.79 prereqs/course (n=33)
ECON: 1.78 prereqs/course (n=104)
ORGB: 1.75 prereqs/course (n=8)
BIOC: 1.73 prereqs/course (n=11)
LING: 1.58 prereqs/course (n=31)
GIS: 1.57 prereqs/course (n=14)
BTC: 1.56 prereqs/course (n=18)
EECE: 1.53 prereqs/course (n=158)
ALY: 1.49 prereqs/course (n=43)
PSY: 1.48 prereqs/course (n=27)
IE: 1.42 prereqs/course (n=74)
INTL: 1.42 prereqs/course (n=48)
EEMB: 1.38 prereqs/course (n=65)
MKTG: 1.38 prereqs/course (n=56)
AFCS: 1.30 prereqs/course (n=47)
PHYS: 1.29 prereqs/course (n=108)
HSV: 1.27 prereqs/course (n=11)
CHEM: 1.25 prereqs/course (n=124)
SOCL: 1.24 prereqs/course (n=79)

```

DADS: 1.23 prereqs/course (n=13)
RGA: 1.23 prereqs/course (n=48)
BIO: 1.22 prereqs/course (n=23)
INMI: 1.22 prereqs/course (n=23)
BIOE: 1.20 prereqs/course (n=75)
LAW: 1.19 prereqs/course (n=207)
MUSC: 1.18 prereqs/course (n=90)
CNET: 1.17 prereqs/course (n=24)
LARC: 1.17 prereqs/course (n=18)
CY: 1.16 prereqs/course (n=44)
CS: 1.15 prereqs/course (n=184)
NRSNG: 1.15 prereqs/course (n=179)
HSCI: 1.14 prereqs/course (n=37)
DEAF: 1.12 prereqs/course (n=8)
BIOL: 1.12 prereqs/course (n=99)
PT: 1.07 prereqs/course (n=100)
CHM: 1.07 prereqs/course (n=15)
MISM: 1.05 prereqs/course (n=38)
FRNH: 1.05 prereqs/course (n=22)
ME: 1.03 prereqs/course (n=96)
DAMG: 1.00 prereqs/course (n=26)
MTH: 1.00 prereqs/course (n=18)
PHTH: 1.00 prereqs/course (n=72)
ARTG: 0.99 prereqs/course (n=74)
CSYE: 0.97 prereqs/course (n=37)
ARTD: 0.97 prereqs/course (n=31)
CIVE: 0.96 prereqs/course (n=136)
ENVR: 0.95 prereqs/course (n=82)
MUST: 0.95 prereqs/course (n=20)
COMM: 0.95 prereqs/course (n=99)
ASNS: 0.94 prereqs/course (n=16)
INTP: 0.94 prereqs/course (n=16)
GAME: 0.93 prereqs/course (n=28)
WMNS: 0.91 prereqs/course (n=43)
PJM: 0.90 prereqs/course (n=61)
EDU: 0.90 prereqs/course (n=129)
PHSC: 0.89 prereqs/course (n=84)
BINF: 0.87 prereqs/course (n=23)
CHME: 0.86 prereqs/course (n=94)
EET: 0.86 prereqs/course (n=21)
HLS: 0.86 prereqs/course (n=14)
GRMN: 0.85 prereqs/course (n=13)
RSSN: 0.83 prereqs/course (n=12)
DS: 0.83 prereqs/course (n=35)
BNSC: 0.82 prereqs/course (n=11)
MKT: 0.82 prereqs/course (n=11)
MATH: 0.81 prereqs/course (n=148)
PORT: 0.79 prereqs/course (n=14)
LW: 0.77 prereqs/course (n=71)

INFO: 0.76 prereqs/course (n=51)
AAI: 0.76 prereqs/course (n=29)
ACC: 0.74 prereqs/course (n=19)
OR: 0.73 prereqs/course (n=15)
ACCT: 0.72 prereqs/course (n=60)
CHNS: 0.71 prereqs/course (n=21)
MSCR: 0.67 prereqs/course (n=52)
ARAB: 0.67 prereqs/course (n=12)
AVM: 0.67 prereqs/course (n=15)
HBRW: 0.67 prereqs/course (n=9)
HSC: 0.67 prereqs/course (n=9)
NETS: 0.67 prereqs/course (n=24)
PTH: 0.67 prereqs/course (n=12)
DGM: 0.65 prereqs/course (n=51)
LDR: 0.64 prereqs/course (n=53)
ENG: 0.62 prereqs/course (n=21)
HMG: 0.62 prereqs/course (n=21)
ITC: 0.61 prereqs/course (n=80)
POLS: 0.61 prereqs/course (n=103)
GET: 0.61 prereqs/course (n=18)
ARTS: 0.60 prereqs/course (n=10)
GE: 0.60 prereqs/course (n=30)
ITLN: 0.59 prereqs/course (n=17)
DGTR: 0.58 prereqs/course (n=33)
PHDL: 0.57 prereqs/course (n=7)
JPNS: 0.55 prereqs/course (n=20)
ARCH: 0.55 prereqs/course (n=71)
JWSS: 0.55 prereqs/course (n=11)
MEIE: 0.54 prereqs/course (n=13)
AMSL: 0.53 prereqs/course (n=15)
EXSC: 0.53 prereqs/course (n=17)
CMN: 0.50 prereqs/course (n=42)
HRMG: 0.50 prereqs/course (n=14)
MUSI: 0.50 prereqs/course (n=36)
PDM: 0.50 prereqs/course (n=8)
PST: 0.50 prereqs/course (n=14)
RMS: 0.50 prereqs/course (n=6)
RPT: 0.50 prereqs/course (n=6)
SIA: 0.47 prereqs/course (n=19)
CET: 0.47 prereqs/course (n=17)
INSH: 0.47 prereqs/course (n=43)
PHMD: 0.46 prereqs/course (n=99)
PHIL: 0.45 prereqs/course (n=82)
FIN: 0.45 prereqs/course (n=20)
CED: 0.44 prereqs/course (n=32)
INS: 0.41 prereqs/course (n=17)
CLTR: 0.40 prereqs/course (n=15)
HRM: 0.40 prereqs/course (n=25)
PMCL: 0.40 prereqs/course (n=5)

MET: 0.39 prereqs/course (n=18)
PHY: 0.38 prereqs/course (n=13)
HIST: 0.38 prereqs/course (n=114)
ENSY: 0.37 prereqs/course (n=19)
INTB: 0.37 prereqs/course (n=30)
GST: 0.36 prereqs/course (n=39)
EDUT: 0.36 prereqs/course (n=59)
NPM: 0.35 prereqs/course (n=20)
TELE: 0.35 prereqs/course (n=23)
HUSV: 0.34 prereqs/course (n=32)
ARTH: 0.33 prereqs/course (n=24)
ENLR: 0.33 prereqs/course (n=9)
IS: 0.33 prereqs/course (n=6)
LPSC: 0.32 prereqs/course (n=19)
ARTE: 0.29 prereqs/course (n=21)
NNMD: 0.29 prereqs/course (n=14)
LWP: 0.28 prereqs/course (n=29)
PHL: 0.27 prereqs/course (n=11)
TELR: 0.27 prereqs/course (n=11)
CAEP: 0.27 prereqs/course (n=130)
THTR: 0.26 prereqs/course (n=68)
BIOT: 0.26 prereqs/course (n=50)
JRNL: 0.25 prereqs/course (n=67)
EDUC: 0.25 prereqs/course (n=8)
INT: 0.25 prereqs/course (n=12)
SUEN: 0.24 prereqs/course (n=17)
MGMT: 0.21 prereqs/course (n=28)
HLTH: 0.21 prereqs/course (n=48)
HINF: 0.21 prereqs/course (n=39)
GSND: 0.20 prereqs/course (n=30)
MATL: 0.20 prereqs/course (n=15)
INAM: 0.19 prereqs/course (n=77)
NTR: 0.19 prereqs/course (n=26)
SBSY: 0.18 prereqs/course (n=11)
SLPA: 0.17 prereqs/course (n=63)
CRIM: 0.16 prereqs/course (n=93)
APL: 0.15 prereqs/course (n=13)
EXRE: 0.15 prereqs/course (n=13)
PA: 0.15 prereqs/course (n=34)
INPR: 0.14 prereqs/course (n=28)
RFA: 0.14 prereqs/course (n=7)
EAI: 0.12 prereqs/course (n=16)
MSCI: 0.12 prereqs/course (n=25)
CRTE: 0.11 prereqs/course (n=9)
KORE: 0.11 prereqs/course (n=9)
PBR: 0.11 prereqs/course (n=9)
PREL: 0.11 prereqs/course (n=9)
TCC: 0.10 prereqs/course (n=10)
INNO: 0.09 prereqs/course (n=44)

ARTF: 0.08 prereqs/course (n=25)
ART: 0.08 prereqs/course (n=13)
EMGT: 0.06 prereqs/course (n=16)
ENTR: 0.06 prereqs/course (n=36)
BUSN: 0.05 prereqs/course (n=59)
PPUA: 0.05 prereqs/course (n=82)
AACE: 0.00 prereqs/course (n=14)
ABRB: 0.00 prereqs/course (n=18)
ABRC: 0.00 prereqs/course (n=33)
ABRD: 0.00 prereqs/course (n=94)
ABRH: 0.00 prereqs/course (n=2)
ABRL: 0.00 prereqs/course (n=14)
ABRS: 0.00 prereqs/course (n=1)
ABRU: 0.00 prereqs/course (n=1)
AFRS: 0.00 prereqs/course (n=6)
ANT: 0.00 prereqs/course (n=6)
ARMY: 0.00 prereqs/course (n=11)
CJS: 0.00 prereqs/course (n=5)
CMG: 0.00 prereqs/course (n=5)
CMMN: 0.00 prereqs/course (n=1)
COOP: 0.00 prereqs/course (n=5)
COP: 0.00 prereqs/course (n=7)
DA: 0.00 prereqs/course (n=7)
ECN: 0.00 prereqs/course (n=6)
ECNM: 0.00 prereqs/course (n=2)
EEAM: 0.00 prereqs/course (n=11)
EEBA: 0.00 prereqs/course (n=17)
EESC: 0.00 prereqs/course (n=14)
EESH: 0.00 prereqs/course (n=8)
ENGR: 0.00 prereqs/course (n=12)
ENVS: 0.00 prereqs/course (n=6)
ESC: 0.00 prereqs/course (n=7)
EXED: 0.00 prereqs/course (n=7)
FSEM: 0.00 prereqs/course (n=7)
GBST: 0.00 prereqs/course (n=2)
HONR: 0.00 prereqs/course (n=13)
HST: 0.00 prereqs/course (n=11)
HSTY: 0.00 prereqs/course (n=1)
INSC: 0.00 prereqs/course (n=14)
LACS: 0.00 prereqs/course (n=8)
LANG: 0.00 prereqs/course (n=6)
LST: 0.00 prereqs/course (n=14)
MATM: 0.00 prereqs/course (n=4)
MECN: 0.00 prereqs/course (n=3)
MGSC: 0.00 prereqs/course (n=13)
MUS: 0.00 prereqs/course (n=7)
PHLS: 0.00 prereqs/course (n=2)
PLSC: 0.00 prereqs/course (n=1)
PMST: 0.00 prereqs/course (n=4)

```
POL: 0.00 prereqs/course (n=9)
SCLY: 0.00 prereqs/course (n=1)
SMFA: 0.00 prereqs/course (n=3)
SMT: 0.00 prereqs/course (n=10)
SOC: 0.00 prereqs/course (n=14)
STRT: 0.00 prereqs/course (n=12)
```

Department with the most prerequisites per course on average: ENGL (3.47)

The top five departments that have the highest number of course prerequisites per course are:

- ENGL: 3.47 prereqs/course (n=114)
- CRWT: 3.33 prereqs/course (n=18)
- ANTH: 3.22 prereqs/course (n=36)
- ENGW: 3.16 prereqs/course (n=19)
- PSYC: 2.49 prereqs/course (n=97)

This actually makes sense; because these departments are more foundation-based and require a strong foundation in various subjects before taking advanced courses. For example, English and Creative Writing courses often build on prior knowledge of literature, writing techniques, and critical analysis skills. Similarly, Anthropology courses may require understanding of basic concepts in social sciences and history; without which, it would be difficult to grasp more complex topics in advanced courses. Psychology courses often build on foundational knowledge of human behavior, cognitive processes, and research methods, which are essential for understanding more specialized areas within the field.

On the other hand, Public Policy and Urban Affairs (PPUA) department, which have the lowest (non-zero) number of prerequisites per course, are more interdisciplinary and may not require as many specific foundational courses. These fields often draw from a variety of disciplines, allowing students to enter advanced courses with a broader range of prior knowledge and experiences.

There are several departments with zero prerequisites per course; some of them like Environmental Science, similar to PPUA, are very interdisciplinary and do not require specific foundational courses. Others are mostly practical like College of Professional Studies (CPS) or Co-Op programs, or very specialized like Music (MUSC) and Theatre (THEA), where courses may focus more on hands-on learning and skill development rather than building on a series of prerequisite courses.

d

```
import networkx as nx

def build_full_graph(all_courses):
    """
    all_courses: list of dicts with keys like
                  {'dept', 'course_id', 'title', 'prereqs': [...]}

    Returns a DiGraph with edges prereq -> course.
```

```

"""
G = nx.DiGraph()
for c in all_courses:
    cid = c["course_id"]
    # keep some metadata on the node
    G.add_node(cid, title=c.get("title"), dept=c.get("dept"))
    for pre in c.get("prereqs", []):
        # include external prereqs even if not in all_courses
        if pre not in G:
            G.add_node(pre)
        G.add_edge(pre, cid)
return G

def longest_prereq_chain(all_courses):
    """
    Returns (chain, is_dag) where chain is a list of course_ids from
    earliest prereq to final course.
    """
    G = build_full_graph(all_courses)

    # G.remove_edges_from(nx.selfloop_edges(G))

    # print a cycle!
    try:
        cycle = nx.find_cycle(G)
        print("Cycle detected:", " -> ".join(n for n, _ in cycle) + "
-> " + cycle[0][0])
    except nx.NetworkXNoCycle:
        print("No cycle found!")

    if nx.is_directed_acyclic_graph(G):
        chain = nx.dag_longest_path(G)
        return chain, True

    # Graph has cycles: use condensation DAG to get an acyclic view
    sccs = list(nx.strongly_connected_components(G)) # list of sets
of nodes
    CG = nx.condensation(G, sccs)
    comp_path = nx.dag_longest_path(CG)

    # pick one representative course_id from each SCC to display
    comp_index_to_nodes = {i: nodes for i, nodes in enumerate(sccs)}
    chain = [next(iter(comp_index_to_nodes[i])) for i in comp_path]
    return chain, False

chain, is_dag = longest_prereq_chain(all_courses)

print("Used DAG longest path" if is_dag else "Cycles detected; used
condensation DAG")
print("Longest prerequisite chain length:", max(0, len(chain) - 1))

```

```

print("Chain:")
print(" -> ".join(chain))

# Optional: show titles alongside IDs
# If you want titles too:
id_to_title = {c["course_id"]: c.get("title","") for c in all_courses}
print("\nWith titles:")
for cid in chain:
    print(f"{cid}: {id_to_title.get(cid,'')}")

Cycle detected: SPNS 3602 -> SPNS 3603 -> SPNS 3602
Cycles detected; used condensation DAG
Longest prerequisite chain length: 9
Chain:
BIOL 1107 -> BIOL 1113 -> PHSC 2301 -> PHSC 2303 -> HSCI 1105 -> NRSG
2220 -> NRSG 3320 -> NRSG 3420 -> NRSG 4502 -> NRSG 4995

With titles:
BIOL 1107: BIOL 1107. Foundations of Biology. (4 Hours)
BIOL 1113: BIOL 1113. General Biology 2. (4 Hours)
PHSC 2301: PHSC 2301. Human Physiology 1. (3 Hours)
PHSC 2303: PHSC 2303. Human Physiology 2. (3 Hours)
HSCI 1105: HSCI 1105. Human Nutrition. (4 Hours)
NRSG 2220: NRSG 2220. Health Assessment and Fundamental Nursing
Skills. (3 Hours)
NRSG 3320: NRSG 3320. Nursing Care of Adults 1. (4 Hours)
NRSG 3420: NRSG 3420. Nursing Care of Adults 2. (4 Hours)
NRSG 4502: NRSG 4502. Nursing Care of the Child. (4 Hours)
NRSG 4995: NRSG 4995. Comprehensive Nursing Practicum. (2 Hours)

```

It's in the Nursing department! This makes sense because nursing education often requires a sequential learning process, where foundational knowledge in biology, physiology, and health sciences is built upon with more specialized nursing courses. The complexity and responsibility associated with nursing practice necessitate a thorough understanding of various subjects, leading to longer chains of prerequisites. The interesting part is that they first have to take courses on adults, and then they can take a course on children.

Side note: There was some cycles in the graph! For example, check out here: <https://catalog.northeastern.edu/course-descriptions/spns/> Courses 3602 and 3603 are prerequisites of each other!

Because of these, I used condensation to make the graph acyclic. I did this by using `nx.strongly_connected_components` function.

e

1) Prerequisite Network

- **Nodes:** Courses

- **Edges (directed):** Prereq => Course
- **What it captures:** Institutional structure; how students are allowed to progress.

Good for:

1. **Pathways & Depth:**
 - Longest chains
 - Minimal paths to reach a target course
2. **Gateways/Bottlenecks:**
 - High out-degree or high betweenness courses that unlock many others
3. **Curriculum Robustness:**
 - Single points of failure
 - Redundant routes
 - Cycles/inconsistencies
4. **Cross-department Reliance:**
 - Which departments' courses enable others

Methods:

- DAG checks
- Topological sort
- Longest path
- Degree/betweenness centrality
- Reachability
- Community detection on directed graphs (reveals tracks/sequences)

Caveat:

- Reflects policy/requirements, not topical content or emphasis.

2) Word Co-occurrence Network

- **Nodes:** Words/terms
- **Edges (undirected, weighted):** Frequency of co-appearance in descriptions
- **What it captures:** Semantic proximity; how topics and skills cluster in language.

Good for:

1. **Topical Structure:**
 - Communities = Themes
 - Hubs = Broad terms
2. **Course Similarity:**
 - Build course–term bipartite graphs
 - Project to course–course similarity
3. **Trend Analysis:**
 - Emerging terms (e.g., AI, ethics) over time
4. **Interdisciplinarity:**
 - Bridging terms linking departments

Methods:

- Weighted graphs
- TF-IDF
- Community detection (Louvain/Leiden)
- Topic modeling
- Embeddings for semantic similarity

- Caveat:
 - Sensitive to preprocessing (stopwords, stemming, n-grams)
- Can be dominated by generic words without weighting.

Key Differences (and How They Complement)

1. **Structure vs. Semantics:**
 - Prereq graph encodes constraints and sequence.
 - Co-occurrence encodes content and themes.
2. **Centrality Meaning:**
 - In prereqs: "Central" = Gatekeeper/unlocker.
 - In co-occurrence: "Central" = Common or bridging concept (not necessarily important structurally).
3. **Use Together:**
 - Identify structural bottlenecks whose descriptions also bridge topics (prime targets for redesign/capstones).
 - Verify that tracks implied by prerequisites align with coherent thematic clusters in the text.

Question 5

a

Dataset Description

I'm using a dataset of Facebook social network of Northeastern University from the network repository: [Network Repository](#).

Each node represents a person, and each edge represents a friendship tie between two people. The graph is undirected because friendship is a mutual relationship. The edges are unweighted, indicating that all friendships are considered equal without any additional attributes like strength or frequency of interaction.

Summary Statistics

- **Nodes:** 13,900
- **Edges:** 381,900
- **Density:** 0.00396411

- **Maximum Degree:** 968
- **Minimum Degree:** 1
- **Average Degree:** 55
- **Assortativity:** 0.0681706
- **Number of Triangles:** 5M
- **Average Number of Triangles:** 362
- **Maximum Number of Triangles:** 21.6K
- **Average Clustering Coefficient:** 0.201149
- **Fraction of Closed Triangles:** 0.128196
- **Maximum k-core:** 44
- **Lower Bound of Maximum Clique:** 11

The average degree of the nodes is 55, indicating that on average, each person has 55 friends in this network. The maximum degree is 968, meaning that the most connected individual has 968 friends.

The graph has a low density of 0.00396411, suggesting that the network is sparse, with relatively few connections compared to the total possible connections. This is typical for social networks, where not everyone is connected to everyone else, especially in a large community like a university.

In 1999, Barabási and Albert introduced a model to explain the emergence of scale-free networks, which are characterized by a power-law degree distribution. In such networks, a few nodes (hubs) have a very high degree, while most nodes have a low degree. This phenomenon is often observed in real-world networks, including social networks like Facebook.

The assortativity coefficient is 0.0681706, indicating a slight tendency for nodes to connect to others with similar degrees.

Citations

1. Rossi, R. A., & Ahmed, N. K. (2015). The Network Data Repository with Interactive Graph Analytics and Visualization. In AAAI. [Link](#)
2. Traud, A. L., Mucha, P. J., & Porter, M. A. (2012). Social structure of Facebook networks. *Phys. A*, 391(16), 4165-4180.
3. Traud, A. L., Kelsic, E. D., Mucha, P. J., & Porter, M. A. (2011). Comparing Community Structure to Characteristics in Online Collegiate Social Networks. *SIAM Rev.*, 53(3), 526-543.
4. Barabási, A.-L., & Albert, R. (1999). Emergence of Scaling in Random Networks. *Science*, 286(5439), 509-512.

```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import mmread
from collections import Counter
import warnings
warnings.filterwarnings('ignore')
```

```

# Load the network (assuming you've already done this)
print("Loading network...")
matrix = mmread('../data/socfb-Northeastern19/socfb-
Northeastern19.mtx')
G = nx.from_scipy_sparse_array(matrix, create_using=nx.Graph())

print(f'Original Network:')
print(f'Number of nodes: {G.number_of_nodes()}')
print(f'Number of edges: {G.number_of_edges()}')
print(f'Average degree: {2 * G.number_of_edges() /
G.number_of_nodes():.2f}')

Loading network...
Original Network:
Number of nodes: 13882
Number of edges: 381934
Average degree: 55.03

```

b

```

# Remove self-loops if any
G.remove_edges_from(nx.selfloop_edges(G))

# Get the largest connected component for better analysis
# check if the graph is connected
if not nx.is_connected(G):
    print("Graph is not connected. Extracting largest connected
component...")
else:
    print("Graph is connected.")
components = list(nx.connected_components(G))
largest_cc = G.subgraph(max(components, key=len)).copy()
print(f'\nLargest connected component:')
print(f'Nodes: {largest_cc.number_of_nodes()}
({100*largest_cc.number_of_nodes()/G.number_of_nodes():.1f}% of
total)')
print(f'Edges: {largest_cc.number_of_edges()}')

# Use the largest connected component for analysis
G = largest_cc

Graph is not connected. Extracting largest connected component...

Largest connected component:
Nodes: 13868 (99.9% of total)
Edges: 381919

# Create a degree-preserving randomized network using the
configuration model
print("\nCreating randomized network (configuration model)...")

```

```

degree_sequence = [d for n, d in G.degree()]
G_random = nx.configuration_model(degree_sequence,
create_using=nx.Graph())

# Remove self-loops and parallel edges from randomized graph
G_random.remove_edges_from(nx.selfloop_edges(G_random))

print(f'\nRandomized Network:')
print(f'Number of nodes: {G_random.number_of_nodes()}')
print(f'Number of edges: {G_random.number_of_edges()}')

# Alternative: Use double edge swap
# G_random = G.copy()
# nx.double_edge_swap(G_random, nswap=10*G.number_of_edges(),
max_tries=100*G.number_of_edges())

# Compute network properties
print("\nComputing network properties...")

# 1. Degree distribution
degrees = [d for n, d in G.degree()]
degrees_random = [d for n, d in G_random.degree()]

# 2. Clustering coefficient distribution
print("Computing clustering coefficients...")
clustering = list(nx.clustering(G).values())
clustering_random = list(nx.clustering(G_random).values())

# 3. Betweenness centrality
print("Computing betweenness centrality...")
if G.number_of_nodes() > 1000:
    # Sample nodes for betweenness calculation
    sample_size = min(500, G.number_of_nodes())
    sampled_nodes = np.random.choice(list(G.nodes()),
size=sample_size, replace=False)
    betweenness = nx.betweenness_centrality(G, k=sample_size)
    betweenness_random = nx.betweenness_centrality(G_random,
k=sample_size)
else:
    betweenness = nx.betweenness_centrality(G)
    betweenness_random = nx.betweenness_centrality(G_random)

betweenness_values = list(betweenness.values())
betweenness_values_random = list(betweenness_random.values())

```

Creating randomized network (configuration model)...

Randomized Network:
Number of nodes: 13882

Number of edges: 379235

Computing network properties...

Computing clustering coefficients...

Computing betweenness centrality...

Create figure for original network

fig1, axes1 = plt.subplots(1, 3, figsize=(15, 5))

fig1.suptitle('Original Network Properties', fontsize=16)

Plot 1: Degree Distribution

ax1 = axes1[0]

degree_counts = Counter(degrees)

degrees_unique = sorted(degree_counts.keys())

counts = [degree_counts[d] for d in degrees_unique]

Log-binning for better visualization

ax1.loglog(degrees_unique, counts, 'bo-', alpha=0.7, markersize=4)

ax1.set_xlabel('Degree (k)', fontsize=12)

ax1.set_ylabel('Count', fontsize=12)

ax1.set_title('Degree Distribution (log-log)', fontsize=13)

ax1.grid(True, alpha=0.3)

Plot 2: Clustering Coefficient Distribution

ax2 = axes1[1]

Create bins for clustering coefficient

bins = np.linspace(0, 1, 31)

ax2.hist(clustering, bins=bins, alpha=0.7, color='green',
edgecolor='black')

ax2.set_xlabel('Clustering Coefficient', fontsize=12)

ax2.set_ylabel('Count', fontsize=12)

ax2.set_title('Local Clustering Coefficient Distribution',
fontsize=13)

ax2.axvline(np.mean(clustering), color='red', linestyle='--',
label=f'Mean: {np.mean(clustering):.3f}')

ax2.legend()

ax2.grid(True, alpha=0.3)

Plot 3: Betweenness Centrality Distribution

ax3 = axes1[2]

Log scale for betweenness

betweenness_nonzero = [b for b in betweenness_values if b > 0]

ax3.hist(betweenness_nonzero, bins=50, alpha=0.7, color='purple',
edgecolor='black')

ax3.set_xlabel('Betweenness Centrality', fontsize=12)

ax3.set_ylabel('Count', fontsize=12)

ax3.set_title('Betweenness Centrality Distribution', fontsize=13)

ax3.set_yscale('log')

ax3.set_xscale('log')

ax3.grid(True, alpha=0.3)

```

plt.tight_layout()
# plt.savefig('original_network_properties.png', dpi=300,
#             bbox_inches='tight')
plt.show()

# Create figure for randomized network
fig2, axes2 = plt.subplots(1, 3, figsize=(15, 5))
fig2.suptitle('Randomized Network Properties (Configuration Model)',
              fontsize=16)

# Plot 1: Degree Distribution (Randomized)
ax1 = axes2[0]
degree_counts_random = Counter(degrees_random)
degrees_unique_random = sorted(degree_counts_random.keys())
counts_random = [degree_counts_random[d] for d in
                  degrees_unique_random]

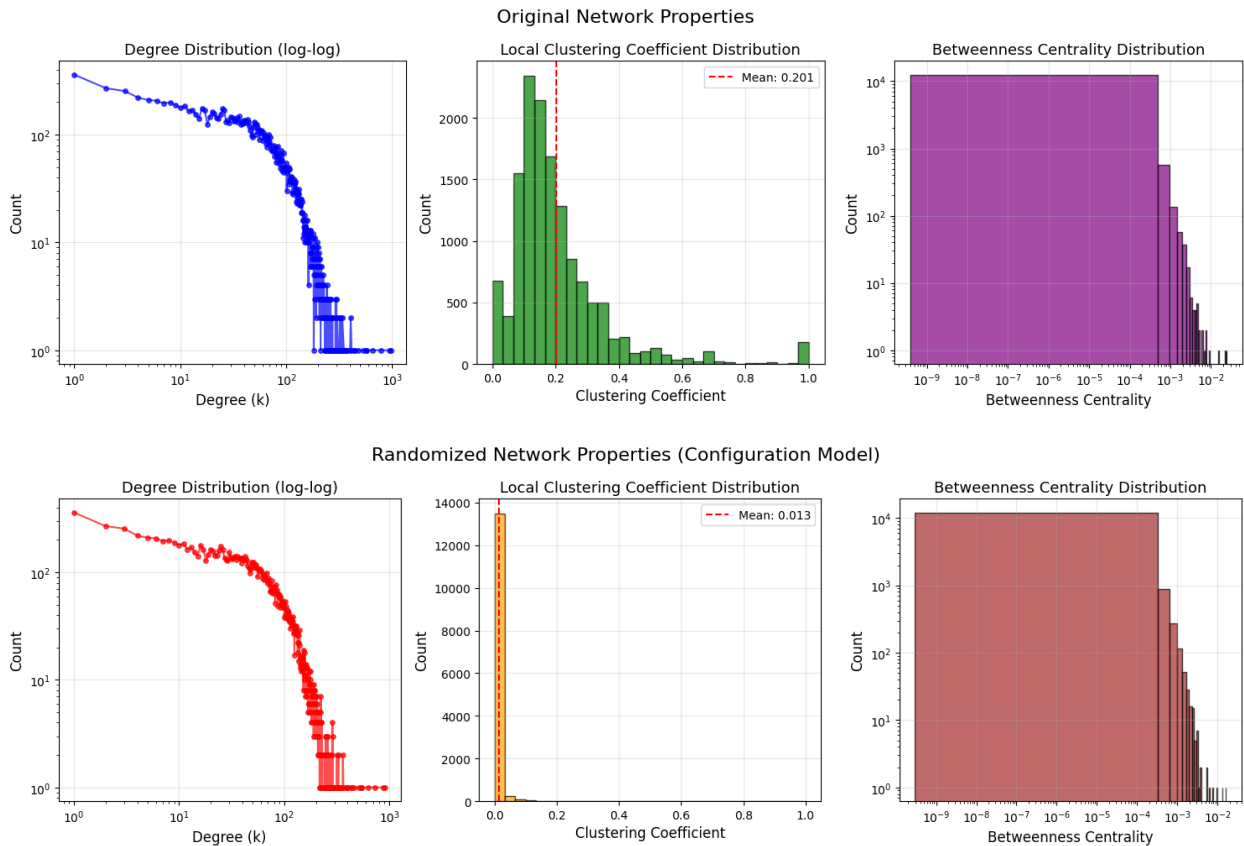
ax1.loglog(degrees_unique_random, counts_random, 'ro-', alpha=0.7,
           markersize=4)
ax1.set_xlabel('Degree (k)', fontsize=12)
ax1.set_ylabel('Count', fontsize=12)
ax1.set_title('Degree Distribution (log-log)', fontsize=13)
ax1.grid(True, alpha=0.3)

# Plot 2: Clustering Coefficient Distribution (Randomized)
ax2 = axes2[1]
ax2.hist(clustering_random, bins=bins, alpha=0.7, color='orange',
         edgecolor='black')
ax2.set_xlabel('Clustering Coefficient', fontsize=12)
ax2.set_ylabel('Count', fontsize=12)
ax2.set_title('Local Clustering Coefficient Distribution',
              fontsize=13)
ax2.axvline(np.mean(clustering_random), color='red', linestyle='--',
            label=f'Mean: {np.mean(clustering_random):.3f}')
ax2.legend()
ax2.grid(True, alpha=0.3)

# Plot 3: Betweenness Centrality Distribution (Randomized)
ax3 = axes2[2]
betweenness_nonzero_random = [b for b in betweenness_values_random if
                              b > 0]
ax3.hist(betweenness_nonzero_random, bins=50, alpha=0.7,
         color='brown', edgecolor='black')
ax3.set_xlabel('Betweenness Centrality', fontsize=12)
ax3.set_ylabel('Count', fontsize=12)
ax3.set_title('Betweenness Centrality Distribution', fontsize=13)
ax3.set_yscale('log')
ax3.set_xscale('log')
ax3.grid(True, alpha=0.3)

```

```
plt.tight_layout()
# plt.savefig('randomized_network_properties.png', dpi=300,
#             bbox_inches='tight')
plt.show()
```



```
# Print comparative statistics
print("\n1. DEGREE DISTRIBUTION:")
print(f"    Original - Mean degree: {np.mean(degrees):.2f}, Std: {np.std(degrees):.2f}")
print(f"    Random   - Mean degree: {np.mean(degrees_random):.2f}, Std: {np.std(degrees_random):.2f}")

print("\n2. CLUSTERING COEFFICIENT:")
print(f"    Original - Mean: {np.mean(clustering):.4f}, Std: {np.std(clustering):.4f}")
print(f"    Random   - Mean: {np.mean(clustering_random):.4f}, Std: {np.std(clustering_random):.4f}")
ratio = np.mean(clustering) / np.mean(clustering_random) if
np.mean(clustering_random) > 0 else float('inf')

print("\n3. BETWEENNESS CENTRALITY:")
print(f"    Original - Mean: {np.mean(betweenness_values):.6f}, Max:
```

```

{np.max(betweenness_values):.6f}")
print(f"    Random    - Mean: {np.mean(betweenness_values_random):.6f},
Max: {np.max(betweenness_values_random):.6f}")

# Additional network metrics
print("\n4. ADDITIONAL METRICS:")
print(f"    Original - Transitivity: {nx.transitivity(G):.4f}")
print(f"    Random    - Transitivity: {nx.transitivity(G_random):.4f}")

print(f"\n    Original - Assortativity:
{nx.degree_assortativity_coefficient(G):.4f}")
print(f"    Random    - Assortativity:
{nx.degree_assortativity_coefficient(G_random):.4f}")

# number of triangles
triangles = sum(nx.triangles(G).values()) // 3
triangles_random = sum(nx.triangles(G_random).values()) // 3
print(f"\n    Original - Number of triangles: {triangles}")
print(f"    Random    - Number of triangles: {triangles_random}")

```

```

1. DEGREE DISTRIBUTION:
    Original - Mean degree: 55.03, Std: 51.77
    Random    - Mean degree: 54.64, Std: 50.88

2. CLUSTERING COEFFICIENT:
    Original - Mean: 0.2011, Std: 0.1557
    Random    - Mean: 0.0130, Std: 0.0219

3. BETWEENNESS CENTRALITY:
    Original - Mean: 0.000146, Max: 0.024282
    Random    - Mean: 0.000131, Max: 0.016402

4. ADDITIONAL METRICS:
    Original - Transitivity: 0.1282
    Random    - Transitivity: 0.0134

    Original - Assortativity: 0.0682
    Random    - Assortativity: -0.0025

    Original - Number of triangles: 1676704
    Random    - Number of triangles: 171553

```

C

Some notes:

1. Degree distribution is preserved (by design)
2. Original graph has 15.51x higher clustering than random network!!

3. Original graph has 12x higher transitivity than random network!! This is expected in social networks, where triadic closure is common.
4. Original graph has positive assortativity, indicating that nodes tend to connect to others with similar degree. Random network is near zero.
5. Original graph has 10x more triangles, reflecting community structure common in social networks.

Structural Interpretations:

Small-World Properties: The original network exhibits classic small-world characteristics; high clustering while maintaining short paths, unlike the random network which has minimal clustering. **Community Structure:** The 15-fold difference in clustering coefficient strongly suggests the presence of dense communities or social groups (e.g., friend circles, study groups, dorm communities) that are absent in random wiring. **Homophily:** Positive assortativity (0.068) in the original network indicates that students tend to connect with others of similar popularity/connectivity, while the random network shows no such preference. **Social Mechanisms at Work:**

Triadic closure: Friends introduce friends, creating triangles **Preferential attachment:** Popular individuals attract more connections **Homophily:** Similar individuals (year, major, interests) connect more frequently **Spatial/institutional constraints:** Dorm proximity, shared classes influence connections

Network Resilience: The presence of clear bridge nodes (high betweenness) in the original network suggests vulnerability to targeted removal but also indicates natural information brokers who connect different social circles.

This analysis confirms that the Northeastern University social network has significant non-random structure driven by social processes that cannot be explained by degree distribution alone. The network exhibits the hallmarks of real social networks: community structure, triadic closure, and assortative mixing patterns.

Question 6

First of all, thank you for asking our opinions and comments about the course. This is highly valuable and I appreciate it a lot. I think the course is well-structured and the content is very interesting. As a PhD student who works with Network Science tools, I believe this course gives me strong skills to work with networkx and get more comfortable with implementing network-related codes.

I particularly like the hands-on approach we have during the sessions, but I think it would be better if, instead of just covering numerous concepts in this course, we focus on some specific topics and go deeper into them. In the last couple of sessions, I learned a lot about networkx; However, with this fast pace it's not very likely for me to keep all those new functions, tools, and concepts in mind, when I really need them.

And this is the comment of a person who is already familiar with coding and Python (but no networkx background). I think for students who are new to coding, it might be even more challenging to keep up with the pace of the course.

There is also another way that comes to my mind about how we can improve the course. It's a slight change in the HW+project setting. I think it would be great if each student can pick a specific large dataset they are interested in, and as we go into the course (more lecture-based) that you talk about different concepts, and going over their implementations in networkx (less detailed than current version), we can apply those concepts to our datasets. Then, students can have 3-4 pitch sessions during the semester to share their progress and get feedback from you and other classmates (i.e., each student picks a large network graph dataset from networkrepository.com). For example, if we learn about centrality measures, we can compute different centrality measures for our graph. If we learn about community detection, we can apply different community detection algorithms to our graph. If we learn about graph visualization, we can visualize our graph using different techniques. So on and so forth.

This way, students can learn the concepts better and also have an ongoing project for the entire semester that keeps them engaged with course materials and motivated to learn more and apply new tools and concepts to their projects as they go over the course. Also, I really love the idea of the sense of ownership that comes with having such a project; as everyone has their own network dataset, and it's THEIRS. So they are more motivated to work on it and learn more about it.